

Marketing Agent Dashboard: Codebase Guide

How the dashboard, its backend and the agent controls fit together, what every part of the code does, and how to run, test and extend it.

Repository	Marketing_Agent (folders dashboard/ and dashboardbackend/)
Frontend	Next.js 15 (App Router), React 19, TanStack Query 5, supabase-js 2
Backend	Python, FastAPI, asyncpg, PyJWT, httpx
Data & auth	Supabase (Postgres + Auth)
Agent control	Pipeline control API on Render, applied by the agent on the office machine
As of	7 October 2026, commit 2a16577

Contents

1. What this project is
2. Architecture
3. Repository layout
4. Running it locally
5. Sign-in, roles and security
6. The pages
7. Frontend internals
8. Dashboard backend (FastAPI)
9. How the data is derived
10. Agent Start / Stop
11. User management API
12. Tests
13. Environment variables
14. Extending the code
15. Known limitations and next steps
16. Deployment (Vercel + Render)
17. Blog Plan (part 2, from page 19)

1. What this project is

This repository is the web dashboard for the AI marketing agent. The agent itself lives in a separate repository (Marketing-AI-Agent) and runs on an office machine: it finds leads, drafts and sends outreach emails, classifies replies, drafts follow-ups and writes blog posts. Everything it produces is stored in Supabase Postgres. People approve emails in Slack.

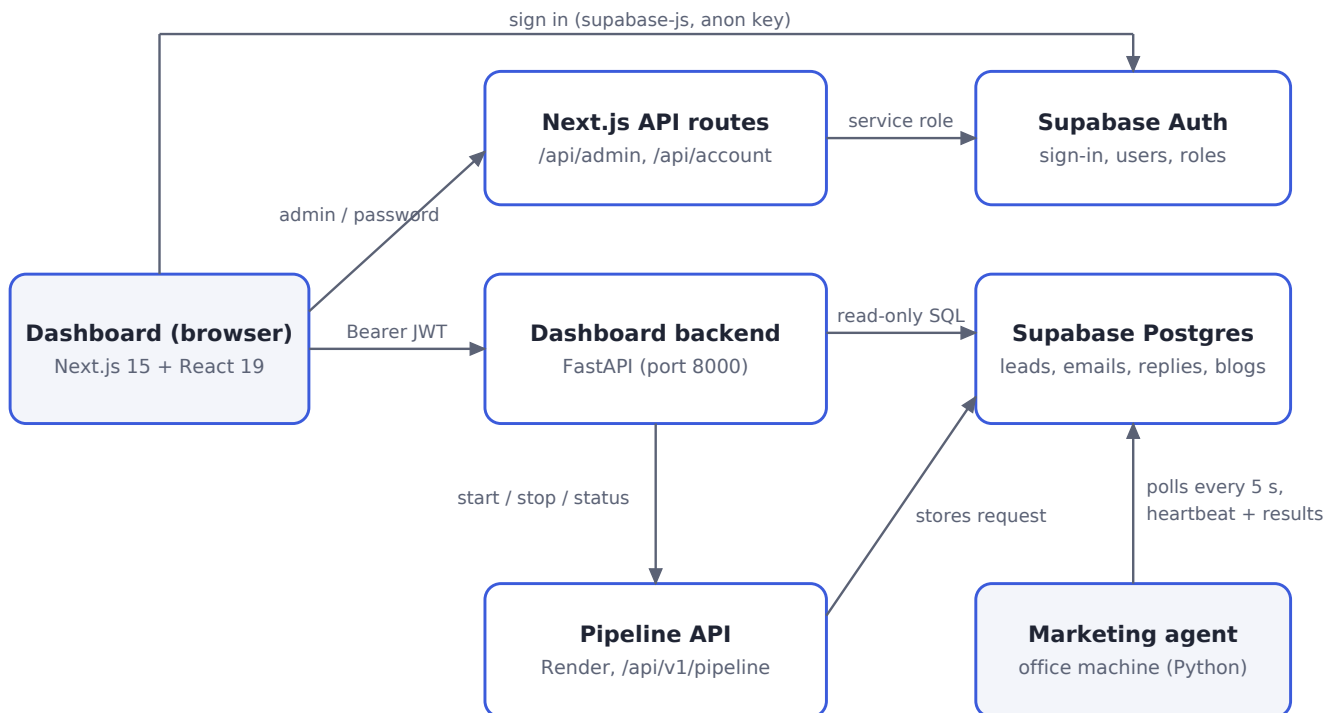
The dashboard lets the team see that work and control the agent:

- **Dashboard:** start or stop the agent, see lead totals, lead sources and the full lead list.
- **History:** every email sent and every reply received.
- **Positive Leads:** leads whose latest reply is interested, uncertain (needs review) or a question, with how long each has been waiting for an answer.
- **Blogs:** blog posts and where they were published.
- **Users** (admins only): create employees, reset passwords, make admins, remove accounts.
- **Profile:** who is signed in, and changing your own password.

The project has two parts that run separately:

Part	Folder	What it does
Dashboard	dashboard/	Next.js app. Renders every page in the browser, signs users in with Supabase Auth, and has a few server-side API routes for user management (they hold the Supabase service-role key).
Dashboard backend	dashboardbackend /	FastAPI service. Verifies the user's Supabase token on every call, reads leads, emails, replies and blogs from Postgres (read-only), and forwards Start/Stop to the pipeline API.

2. Architecture



Four kinds of traffic leave the browser:

- **Sign-in** goes straight to Supabase Auth through supabase-js with the public anon key. The session (access token + refresh token) is kept by supabase-js in the browser.
- **Data** (leads, emails, replies, blogs, agent status) goes to the FastAPI backend with `Authorization: Bearer <access token>`. The backend verifies the token, then runs SQL against Postgres.
- **User management and password changes** go to the dashboard's own Next.js API routes. They run on the server with the Supabase service-role key, which never reaches the browser.
- **Agent Start/Stop** goes to the FastAPI backend, which calls the pipeline API on Render with a secret token. The office machine is behind the office router, so nothing can call it directly: the pipeline API stores the request in the database and the agent polls for it every 5 seconds.

Design choices worth knowing:

- The FastAPI backend never writes to the agent's tables. The agent's services are the only writers.
- Roles live in Supabase `app_metadata`, which only the server (service role) can change. Users can edit their own `user_metadata` (their name), but that is never trusted for permissions.
- Every page is a client component. Pages are guarded in the browser for convenience, but every API checks the token again, so the guard is not the security boundary.

3. Repository layout

Path	Purpose
dashboard/app/layout.js	Root HTML: fonts, page title template, and a tiny script that applies the saved light/dark theme before first paint.
dashboard/app/page.js	Redirects / to /dashboard.
dashboard/app/login/page.js	Sign-in page (renders LoginView).
dashboard/app/(app)/layout.js	Wraps every signed-in page in AppShell (auth guard, top bar, data and drawer providers).
dashboard/app/(app)/error.js	Error boundary: a crash inside a page shows a “Something went wrong” card and keeps the top bar.
dashboard/app/(app)/*/page.js	One thin file per page (dashboard, history, positive-leads, blogs, users, profile) that sets the title and renders its View component.
dashboard/app/api/	Server-only routes: admin/users, admin/users/[id], account/password.
dashboard/components/	All UI: one *View per page, AppShell, DataProvider, DrawerProvider + drawerKinds, ConfirmDialog, shared ui.js pieces, Icon, FormattedContent.
dashboard/core/api.js	Every call to the FastAPI backend (apiFetch + one function per endpoint).
dashboard/core/auth.js	Everything about sign-in: session, isAdmin, signIn/signOut, authFetch for the app's own /api routes.
dashboard/core/admin.js	Client calls for the admin user routes.
dashboard/core/format.js	Date formatting, badge colours (toneOf), source labels, reply-label names.
dashboard/core/supabase.js	The single browser Supabase client (URL + anon key).
dashboard/core/server/supabaseAdmin.js	Server-only helpers: service-role client, getCaller, requireAdmin, error wrapper.
dashboard/app/globals.css	All styling, including light and dark themes via data-theme on <html>.
dashboardbackend/app/main.py	FastAPI app: CORS, /health, /auth/me, includes the data and agent routers.
dashboardbackend/app/auth.py	Verifies Supabase JWTs (JWKS or HS256 secret) and rejects removed users.
dashboardbackend/app/data.py	Read-only SQL routes for leads, emails, replies, blogs.
dashboardbackend/app/agent.py	Start/Stop/status, forwarded to the pipeline API.
dashboardbackend/tests/	pytest suites for auth, data and agent routes.
dashboardbackend/sample_data.sql	Sample rows (all *.example domains) to try the dashboard; includes its own cleanup.

4. Running it locally

Backend (Python 3, from dashboardbackend/):

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
# create .env with the variables in section 13
uvicorn app.main:app --reload --port 8000
pytest
```

Dashboard (Node 18+, from dashboard/):

```
npm install
# create .env.local with the variables in section 13
npm run dev          # http://localhost:3000
```

Notes:

- Do not run `npm run build` while `npm run dev` is running: both write to `.next` and the dev server starts returning 500 errors.
- uvicorn's `--reload` only watches Python files. After editing `.env`, restart the backend.
- The backend creates its Postgres connection pool on the first request, so a wrong `DATABASE_URL` shows up as an error on the first data call, not at startup.
- To see data without the agent, run `sample_data.sql` in the Supabase SQL editor (it needs the agent's migrations 0002, 0005 and 0006).

5. Sign-in, roles and security

Signing in

- LoginView calls `supabase.auth.signInWithPassword`. There is no self-service sign-up or password reset: an admin creates accounts.
- AppShell (around every signed-in page) reads the session. No session: redirect to `/login`. Otherwise the page renders immediately and `GET /auth/me` runs in the background; if the backend rejects the token the user is signed out, and if the backend is unreachable an “offline” card with Retry is shown.
- The admin flag in the browser comes from `session.user.app_metadata.role`. It only decides whether the Users tab is shown.

How the backend verifies a token (`auth.py`)

- Asymmetric tokens (ES256, RS256, EdDSA) are checked against the project's JWKS at `<SUPABASE_URL>/auth/v1/.well-known/jwks.json` (keys cached). Legacy HS256 tokens use `SUPABASE_JWT_SECRET`.
- The audience must be authenticated and `exp` and `sub` are required.
- A token whose `app_metadata.removed` is `true` is refused at once, even before it expires.
- Every data and agent route depends on `current_user`, so a missing or bad token gets 401.

Roles

Role	Can do
employee (default)	See every page except Users; start/stop the agent; change their own password.
admin	Everything an employee can, plus: create users, set their passwords, make an employee an admin (or remove admin rights), remove users. Admins cannot change their own role or remove themselves, so at least one admin always remains.

The Next.js API routes re-check the caller on every request: `getCaller` asks Supabase for the user behind the token (with the service-role key), and `requireAdmin` reads the role from that fresh record. A role change therefore takes effect on the server immediately; only the Users tab waits for the person's token to refresh (within an hour or on next sign-in).

Removing a user

Nothing is deleted. The account gets `app_metadata.removed = true` (hidden from the list) and a ban of about 100 years (no sign-in, no token refresh). Both the FastAPI backend and the Next.js routes refuse removed users.

One setting outside the code

In Supabase, **Authentication → Sign In / Providers → “Allow new users to sign up”** must be off. If it is on, anyone holding the public anon key can create an account directly in Supabase and read the dashboard's data as an employee (they still cannot make themselves an admin).

6. The pages

Page	What it shows	Data (backend route)
Dashboard	Agent Control card (Start/Stop, state, current step, last pass, test mode). Lead stat cards (total, awaiting, responded), a “Leads by Source” bar, and the lead table with status and source filter chips.	/agent, /leads/stats, /leads
History	Two tables side by side: emails sent and replies received, each with a total.	/emails/sent, /emails/replies
Positive Leads	Stat cards (positive leads, leads replied, positive rate). Three groups: Positive, Needs review, Questions. The group is kept in the URL (?group=review). Search by name, company or reply text; sort by follow-up, newest, oldest or name. Follow-up column: “We replied”, “Answer queued” or “Waiting N days” (red after 3 days). A note shows replies still being classified.	/leads/positive
Blogs	Total, posted and not-posted counts, and the list of posts with the site each went to.	/blogs
Users	Admins only. Add-employee form (name, email, password), and the user list with Make admin / Remove admin, Reset password and Remove. Non-admins who open the URL see “Only admins can manage users.”	Next.js /api/admin/users
Profile	Name, email and role badge; change-password form (current, new, confirm).	Supabase session; Next.js /api/account/password
Sign in	Email and password. “Forgot your password? Contact your admin.”	Supabase Auth

The detail drawer

Clicking any row (or pressing Enter on it) opens one shared drawer on the right. It shows the item's details, Previous/Next buttons that step through exactly the rows currently visible (respecting filters, search and sort), and links between related items: a lead opens its email conversation, a sent email opens its reply, a reply opens the original email. Those links switch page first when needed. Esc closes it; focus stays inside while it is open and returns to the row afterwards.

7. Frontend internals

AppShell

The auth guard and frame for every signed-in page: checks the session, renders the top bar (tabs, theme toggle, sign-out), and wraps the page in `DataProvider` and `DrawerProvider`.

DataProvider: data loading and caching

- Built on TanStack Query. Each kind of data is a “section”: agent, stats, leads, sent, replies, blogs, positive. Each maps to one function in `core/api.js`.
- A page calls `useSection('leads')` and gets `{ status: 'loading' | 'ready' | 'error', data, reload }`. Sections load on first use, are shared between pages, and refetch in the background after 30 s (on window focus or reconnect). Failed requests retry once.
- Data already on screen stays visible if a background refetch fails.
- The agent section polls: every 3 s while a Start/Stop is being applied, otherwise every 10 s.
- It also holds UI state that must survive tab switches: the lead filters and the Positive Leads search and sort.

DrawerProvider and drawerKinds

`DrawerProvider` owns the single drawer: which kind and id are open, loading the detail (cached per item), Previous/Next, keyboard handling and cross-page navigation (`goto(page, kind, id)`). `drawerKinds.js` describes what each kind shows: lead, positiveLead, reviewLead, questionLead, sent, reply, blog. Each kind names the section behind Previous/Next, how to load one item, and how to turn it into rows, an action button and a body.

Shared pieces

File	Contents
<code>ui.js</code>	Badge, StatusBadge, SourceCell, SiteCell, Skeleton, ErrorBox, EmptyBox, CardHead, MetaRow, ViewCell, ExtLink, StatCards, and DataTable (skeleton / error / empty / table in one).
<code>ConfirmDialog.js</code>	<code>useConfirm()</code> : a styled replacement for <code>confirm()/prompt()</code> built on <code><dialog></code> . Returns true/false, or the typed password.
<code>FormattedContent.js</code>	Minimal formatter for email and blog bodies: headings, lists, quotes, paragraphs, and clickable URLs and emails. No HTML from the data is ever injected.
<code>Icon.js</code>	Inline SVG icons by name; unknown names render nothing.
<code>OpenRow.js</code>	Table row that opens the drawer and shows itself as selected.
<code>core/format.js</code>	<code>fmtDate</code> (all times shown in UTC), <code>daysSince</code> , <code>toneOf</code> (badge colours by text), <code>SOURCES</code> , <code>REPLY_LABELS</code> .

Errors from the backend: `apiFetch` throws an `ApiError` with the HTTP status (0 when the server can't be reached) and, when FastAPI sends one, its detail message.

8. Dashboard backend (FastAPI)

Route	Auth	Returns
GET /health	none	{ ok: true }
GET /auth/me	Bearer	id, email, role of the caller
GET /leads	Bearer	All leads, newest first, without contact details. Optional ?status=awaiting responded and ?source=linkedin x other.
GET /leads/stats	Bearer	total, awaiting, responded, bySource
GET /leads/positive	Bearer	positive, review, questions (each ordered for follow-up), repliedLeads, pending
GET /leads/{id}	Bearer	One lead with title, company, URLs, email, the conversation to open, latest reply label and review reason
GET /emails/sent	Bearer	Sent and bounced emails, newest first (no body)
GET /emails/sent/{id}	Bearer	One sent email including its body
GET /emails/replies	Bearer	Replies, newest first (no body)
GET /emails/replies/{id}	Bearer	One reply with body and the original email it answers (or null)
GET /blogs	Bearer	Blog posts, newest first (no content)
GET /blogs/{id}	Bearer	One post including its Markdown content
GET /agent	Bearer	Agent status from the pipeline API
POST /agent/start	Bearer	Requests a start; records the caller's email
POST /agent/stop	Bearer	Requests a stop; records the caller's email

- **Response shape.** Database columns are snake_case; `_plain()` turns each row into camelCase JSON (sent_at → sentAt), with UUIDs and timestamps as strings. Agent responses are camelCased the same way.
- **Numbers.** Leads, emails, replies and blogs get a stable 1-based number (oldest = 1) from `row_number()`, so the UI can show “#012”.
- **Bad ids.** An id that isn't a UUID returns 404, not 500.
- **Connection pool.** `asyncpg` pool (1–5 connections) created on first use with `statement_cache_size=0`, which the Supabase transaction pooler (pgbouncer) requires.
- **CORS.** Origins from `CORS_ORIGINS` (default `http://localhost:3000`); GET and POST only.

9. How the data is derived

All of this is SQL in `dashboardbackend/app/data.py` over the agent's tables `companies`, `contacts`, `emails`, `replies` and `content_posts`.

Concept	Rule
Lead	A <code>contacts</code> row joined to its <code>companies</code> row. Name falls back to the email, then the company name.
Lead source	<code>linkedin</code> if <code>companies.source</code> contains "linkedin"; <code>x</code> if it contains the word "x" or "twitter"; otherwise <code>other</code> .
Lead status	Responded if the contact has any reply that is not out-of-office or a bounce; otherwise Awaiting .
Sent email	An <code>emails</code> row with status <code>sent</code> (shown as <code>Delivered</code>) or <code>bounced</code> (shown as <code>Failed</code>) and a sent time.
Reply subject	"Re: " + the subject of the email it answers.
Blog status	<code>published</code> → <code>Posted</code> ; anything else → <code>Not Posted</code> . Site: <code>Dev.to</code> , <code>WordPress</code> or <code>LinkedIn</code> , depending on which URL is set.

Positive Leads

- Each lead is judged on its **latest classified reply written by a person** (out-of-office and bounces are skipped). A later "not interested" replaces an earlier "interested".
- **Needs review**: the classifier recorded low confidence (review_note like "confidence 0.42 < 0.6") and the label is `interested`, `question` or `not_interested`. A doubtful "no" may be a hidden "yes".
- **Positive**: label `interested` with confidence. **Questions**: label `question`.
- **Follow-up**: "replied" once an answer to that reply was sent, "queued" if one is approved or sending, otherwise "waiting".
- **Order**: waiting leads first, longest-waiting on top; then the rest, newest reply first.
- **Preview**: the reply text without quoted mail and without a greeting line, on one line, up to 100 characters (the same rules as the agent's reply classifier).
- **pending**: replies not classified yet; the page says they will appear shortly.

10. Agent Start / Stop

The agent runs on the office machine and nothing on the internet can reach it, so control goes through the pipeline API:

```
Dashboard -> POST /agent/start (dashboard backend, checks the user's token)
           -> POST <AGENT_API_URL>/api/v1/pipeline/start Authorization: Bearer
           <AGENT_API_TOKEN>
               body: { "requested_by": "<user email>" }
           -> pipeline API stores the request in Supabase
           -> agent polls every 5 s, applies it, reports state + heartbeat
```

- Start and Stop are **requests**. They return at once; the agent applies them within about 5 s (a stop can take up to 30 s while the current step winds down). The dashboard polls status every 3 s until inSync is true.
- While running, the agent does passes 30 minutes apart: Gmail inbox → replies → send approved → follow-ups → lead discovery → blog (at most one post a day). Emails approved in Slack go out within about 20 s.
- While stopped, nothing is searched, drafted, sent or polled. Slack approvals are still recorded and go out after the next start.
- If the office machine is off, a request is kept and applied when it comes back.
- The token is only in the backend's environment; the browser never sees it.

What the card shows

Condition	Badge	Button and note
Office machine not reporting (heartbeat older than 60 s)	Offline	"Office machine offline." plus "It will start when it comes back." if Start was requested. The button still sends requests.
Request not applied yet	Starting... / Stopping...	Button disabled until in sync.
Running	Running	Stop button; the current step (e.g. "Finding leads...") or when the next pass starts.
Stopped	Stopped	Start button.

Under the note the card also shows the last pass (Succeeded, Failed or Stopped, with time), who last started or stopped the agent, "Email sending is off." when sending is disabled, and "Test mode: emails go to the test inbox."

Errors are explained: a missing token gives 503 "AGENT_API_TOKEN is not set", a rejected token or an unreachable API gives 502 with the reason. The HTTP timeout is 60 s because a sleeping Render instance can take close to a minute to wake.

11. User management API (Next.js routes)

These run on the Next.js server with `SUPABASE_SERVICE_ROLE_KEY` (imported with `server-only` so it can never be bundled for the browser). The browser calls them with `authFetch`, which adds the user's token. Any unexpected failure becomes a JSON 500 with a generic message.

Route	Who	What it does
GET <code>/api/admin/users</code>	admin	Lists users (up to 200), hiding removed ones: id, email, name, role, created and last sign-in times.
POST <code>/api/admin/users</code>	admin	Creates a confirmed user from name, email and a password the admin chooses (8+ characters). 409 if the email already has an account.
PATCH <code>/api/admin/users/{id}</code>	admin	<code>{ password }</code> sets a new password; <code>{ role: 'admin' 'employee' }</code> changes the role. Not allowed on yourself.
DELETE <code>/api/admin/users/{id}</code>	admin	Marks the user removed and bans them. Not allowed on yourself.
POST <code>/api/account/password</code>	any user	Checks the current password by signing in with it, then sets the new one (8+ characters, different from the current one).

12. Tests

Backend tests run with `pytest` from `dashboardbackend/` (19 tests, all passing). They need no database or network: tokens are signed with a test secret and the database and pipeline API are replaced with fakes.

File	Covers
<code>tests/test_auth.py</code>	Public <code>/health</code> ; valid, missing, expired, wrong-audience and badly signed tokens; removed users are rejected.
<code>tests/test_data.py</code>	Data routes need a login; lead filters hide contact details; lead stats; Positive Leads grouping, follow-up order and preview text.
<code>tests/test_agent.py</code>	Agent routes need a login; Start forwards the token and the user's email; status and stop hit the right URLs; a rejected token is explained; a missing token gives 503.

The dashboard has no automated tests yet, and the SQL queries are not tested against a real database.

13. Environment variables

Names only; the values live in `dashboardbackend/.env` and `dashboard/.env.local`, which are not committed.

Variable	Where	Purpose
NEXT_PUBLIC_SUPABASE_URL	dashboard	Supabase project URL (public). The app refuses to start without an http(s) URL.
NEXT_PUBLIC_SUPABASE_ANON_KEY	dashboard	Supabase anon key (public).
NEXT_PUBLIC_API_URL	dashboard	FastAPI backend URL. Default <code>http://localhost:8000</code> .
SUPABASE_SERVICE_ROLE_KEY	dashboard (server only)	Used by the /api routes for user management. Secret.
SUPABASE_URL	backend	Supabase URL, for JWKS (falls back to NEXT_PUBLIC_SUPABASE_URL).
SUPABASE_JWT_SECRET	backend	Only for legacy HS256 projects.
DATABASE_URL	backend	Pooled Postgres connection string. Secret.
CORS_ORIGINS	backend	Comma-separated dashboard origins. Default <code>http://localhost:3000</code> .
AGENT_API_TOKEN	backend	Pipeline API token (PIPELINE_API_TOKEN is accepted too). Secret.
AGENT_API_URL	backend	Pipeline API base URL. Default <code>https://marketing-agent-pipeline-api.onrender.com</code> .

14. Extending the code

A new data route. Write the query in `app/data.py` and return it through `_rows()` or `_one()` (the router already requires a login). Add a one-line function in `core/api.js`, then either register it as a section in `DataProvider.js` (FETCHERS and EMPTY) or call it directly. Add a test with `_rows` monkeypatched, as in `tests/test_data.py`.

A new page. Add `app/(app)/<name>/page.js` that sets `metadata.title` and renders a `View` component, add the tab to `TABS` in `AppShell.js`, and build the view from `useSection`, `DataTable` and `OpenRow`.

A new kind in the drawer. Add an entry in `drawerKinds.js` with `section`, `list`, `load` and `describe`, then use `<OpenRow kind="...">` in the table.

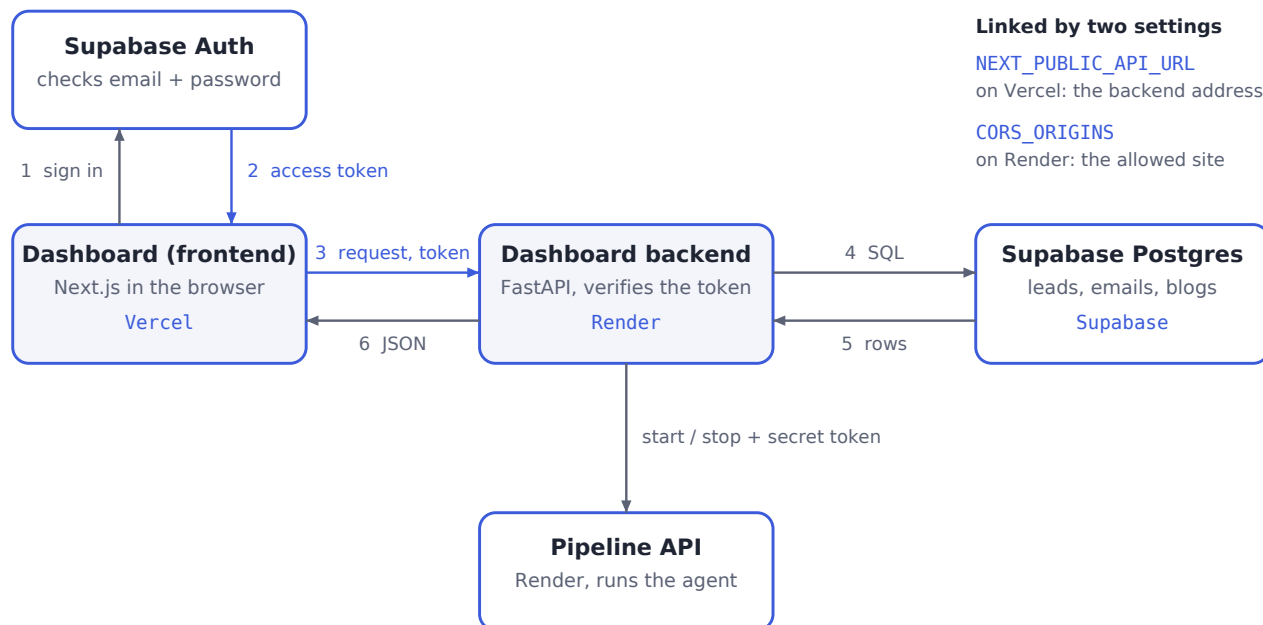
Admin-only server logic. Start the route handler with `requireAdmin(request)` and wrap it in `handle()`.

15. Known limitations and next steps

Area	Issue	Suggestion
Scale	Every list route loads the whole table and filtering happens in Python; there is no pagination. <code>/leads/{id}</code> numbers every contact to return one row.	Add pagination and filter in SQL; store stable numbers in a column.
Users	The user list reads at most 200 users.	Page through Supabase's admin list.
Security	Supabase public sign-up must be off (section 5). Any signed-in user can start or stop the agent.	Verify the setting; restrict Start/Stop to admins if wanted.
Backend	The connection pool is created lazily (two first requests at once can create two) and is never closed.	Create and close it in a FastAPI lifespan handler.
Tests	No dashboard tests; SQL not tested against a database.	Run the queries against <code>sample_data.sql</code> in CI.
Docs	Some comments and the backend README still mention <code>eval/</code> paths and <code>pipeline-api.md</code> , which was removed from this repo (it is in the agent repo). <code>Dashboard-Guide.pdf</code> was deleted in commit <code>f874538</code> .	Update the references.
Dependencies	<code>pytest</code> and <code>httpx</code> are in the production requirements; versions are not pinned.	Split into <code>requirements-dev.txt</code> and <code>pin</code> .
UI	The show/hide password field is repeated in three components and the 8-character minimum in several places. Lead notes are always empty (the backend has no notes yet). All times are shown in UTC.	One <code>PasswordInput</code> component and one constant.
Agent	Start/Stop works end to end, but no full pass had completed when this guide was written.	Run a full pass in test mode and check "Last pass".

16. Deployment (Vercel + Render)

In production the project runs on three hosted services. The dashboard is on **Vercel**, the FastAPI backend is on **Render** (next to the pipeline API), and sign-in plus the database stay on **Supabase**. The browser talks to Supabase Auth and to the backend; only the backend talks to Postgres and the pipeline API.



The token from step 2 is what the backend checks in step 3. The browser never reaches the database or the pipeline directly. User management (section 11) is the one exception to this picture: those routes run on Vercel's server and call Supabase with the service-role key.

How a request travels

1. The user signs in on the dashboard. Email and password go straight to Supabase Auth.
2. Supabase returns an access token, a signed pass that proves who the user is.
3. To load data the dashboard calls `NEXT_PUBLIC_API_URL` through `apiFetch()` (`core/api.js`) with `Authorization: Bearer <token>`.
4. The backend checks the token's signature and expiry (`app/auth.py`); a bad or missing token gets 401. A valid one lets it run SQL over `DATABASE_URL`.
5. Postgres returns the rows.
6. The backend sends them back as JSON. Start and Stop are forwarded to the pipeline API with `AGENT_API_TOKEN`, which never reaches the browser.

The two settings that link frontend and backend

Where	Setting	Value
Vercel (dashboard)	NEXT_PUBLIC_API_URL	The Render backend URL, e.g. <code>https://<backend>.onrender.com</code> . https, no trailing slash.
Render (backend)	CORS_ORIGINS	The Vercel site URL, e.g. <code>https://<app>.vercel.app</code> . Comma-separate extra origins such as <code>http://localhost:3000</code> .

If NEXT_PUBLIC_API_URL is missing, core/api.js falls back to http://localhost:8000. That works in local development and fails for every visitor in production, which shows up as “Cannot reach the server” after login.

Backend on Render

Render → New → Web Service, connected to the GitHub repo:

Setting	Value
Root Directory	dashboardbackend
Runtime	Python 3
Build Command	pip install -r requirements.txt
Start Command	uvicorn app.main:app --host 0.0.0.0 --port \$PORT
Health Check Path	/health
Environment	SUPABASE_URL, DATABASE_URL (pooled connection string), AGENT_API_URL, AGENT_API_TOKEN, CORS_ORIGINS; SUPABASE_JWT_SECRET only for legacy HS256 projects.

Check it by opening <https://<backend>.onrender.com/health>; it returns {"ok": true}.

Dashboard on Vercel

Import the repo in Vercel with **Root Directory** dashboard (framework Next.js, default build and output settings). Add the variables under Project → Settings → Environments → Production:

Variable	Type	Value
NEXT_PUBLIC_API_URL	Config	Render backend URL
NEXT_PUBLIC_SUPABASE_URL	Config	Supabase project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	Config	Public anon key (safe in the browser)
SUPABASE_SERVICE_ROLE_KEY	Secret	Full-access key, used only by the server routes in section 11

NEXT_PUBLIC_* values are baked into the build. After adding or changing one, open Deployments → ... → **Redeploy**; a saved variable does nothing until the next build.

In Supabase → Authentication → URL Configuration, set the Site URL and add a Redirect URL for the Vercel address so sign-in and password-reset links point at production.

Sleep and cold starts

- Render's free plan stops a service after about 15 minutes without traffic. The next request waits 30–60 seconds while it starts again.
- An external monitor (UptimeRobot or cron-job.org) calling /health every 5–10 minutes keeps the backend awake. One always-on service fits in the free plan's ~750 instance hours a month; two do not.
- Vercel does not sleep. Pages come from its CDN and server routes start in well under a second. The Hobby plan is for non-commercial use; business use needs Pro.

Troubleshooting “Cannot reach the server”

This message comes from `apiFetch()` when the request to the backend fails outright. Open the browser's DevTools and match what you see:

What you see	Cause and fix
Requests go to <code>localhost:8000</code>	<code>NEXT_PUBLIC_API_URL</code> not set on Vercel, or set without redeploying.
blocked by CORS policy	<code>CORS_ORIGINS</code> on Render does not exactly match the site URL (scheme, no trailing slash, preview URLs differ). A crashing backend also shows as CORS, so check Render's logs too.
Mixed Content	<code>NEXT_PUBLIC_API_URL</code> uses <code>http</code> instead of <code>https</code> .
Timeout or very slow first load	Backend was asleep. Open <code>/health</code> and retry.
401 on every call	Backend <code>SUPABASE_URL</code> points at a different Supabase project.
500 on data routes	Wrong <code>DATABASE_URL</code> ; the Render logs show the error.

Blog Plan

How we plan weekly blogs on AOSP, HAL, BSP and embedded engineering: the JSON file, uploading it on the Blogs page, and how it is stored in Supabase.

WHERE

Dashboard →
Blogs

STORED IN

`public.blog_plan`

MIGRATION

`002_blog_plan.sql`

WHO CAN UPLOAD

Admins

IN SHORT

1

Write the plan

A JSON file with one brief per blog and its publish date.

2

Upload and check

On the Blogs page. Problems are listed before anything is saved.

3

Scheduled

Saved in Supabase; the page shows which blog goes out on which day.

CONTENTS

1 What the blog plan is

2 The JSON file

3 Uploading a plan

4 What the Blogs page shows

5 How it is stored in Supabase

6 Database reference

7 Table schemas

8 Uploading again

9 What is not automatic yet

10 Testing it

1 What the blog plan is

The blog plan is the list of blogs we want to publish and when. Each week we write it as a JSON file and upload it on the dashboard's **Blogs** page. The dashboard checks the file, saves it in Supabase and shows which blog goes out on which day.

Each entry is a brief, not the blog. It says what the blog is about and how to write it: topic, keywords, tone, length, target Android versions and reference links. The blog itself is written from the brief later (see section 9).

2 The JSON file

One file holds a list of blogs under `"blogs"`. Up to 60 blogs per file, up to 1 MB.

```
{
  "blogs": [
    {
      "id": "BLG-001",
      "topic": "AIDL vs HIDL: Writing a modern HAL in AOSP",
      "category": "HAL",
      "keywords": ["AIDL", "HIDL", "Treble"],
      "publish_date": "2026-10-12",
      "tone": "technical, beginner-friendly",
      "length": "1000-1500 words",
      "target_versions": ["Android 15"],
      "reference_urls": ["https://source.android.com/docs/core/architecture/aidl"]
    }
  ]
}
```

FIELD	REQUIRED	RULES
<code>id</code>	Required	Unique in the file. Up to 40 letters, digits, <code>-</code> or <code>_</code> . Uploading the same id again updates that blog.
<code>topic</code>	Required	10 to 200 characters.
<code>category</code>	Required	Usually AOSP, HAL, BSP, Embedded, Kernel, Drivers, Security or Tooling. Others are allowed with a warning.
<code>keywords</code>	Required	1 to 10 words or phrases.
<code>publish_date</code>	Required	<code>YYYY-MM-DD</code> , today or later.
<code>tone</code>	Optional	Text, up to 100 characters.
<code>length</code>	Optional	Text, up to 100 characters, e.g. <code>"1000-1500 words"</code> .
<code>target_versions</code>	Optional	List of texts, e.g. <code>["Android 15", "Android 16"]</code> .
<code>reference_urls</code>	Optional	Up to 10 links, each starting with <code>http://</code> or <code>https://</code> .

3 Uploading a plan

Only admins see the upload button. The backend checks the admin role in the database on every upload, so the button being visible is not what allows it.

- 1 Open **Blogs** and click **Upload blog plan** at the top of the Blog Plan card.
- 2 Choose the `.json` file. The dashboard checks it first and **saves nothing yet**.
- 3 A preview lists every blog with its date and what will happen to it.
- 4 Fix any problems in the file and upload it again; **Import** stays disabled while there are problems. Warnings (for example two blogs on the same day) don't stop the import.
- 5 Click **Import**. The page shows *"Imported: 4 new, 1 updated."* and the schedule refreshes.

THE PREVIEW

DATE	ID	TOPIC	CATEGORY	WILL BE
Mon, Oct 12	BLG-001	AIDL vs HIDL: Writing a modern HAL in AOSP	HAL	New
Thu, Oct 15	BLG-002	Bringing up a new board: a BSP checklist	BSP	Update
Mon, Oct 19	BLG-003	What changes for vendors in Android 16 GKI	Kernel	Skipped (posted)

What the preview means

- **New** not in the plan yet.
- **Update** a blog with this id is already in the plan; its details are replaced.
- **Skipped (posted)** already posted, so it is never changed.

How problems look

BLG-102 • publish_date: Use a real date written as YYYY-MM-DD.

BLG-104 • reference_urls: Give at most 10 links, each starting with `http://` or `https://`.

4 What the Blogs page shows

BLOG PLAN CARD (AFTER UPLOADING THE SAMPLE FILE)

Planned blogs

6

This week

0

Posted

0

Next blog

Mon, Oct 12
AIDL vs HIDL...

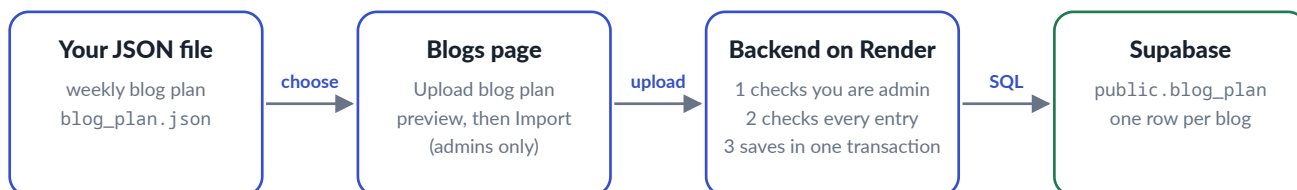
DATE	BLOG	CATEGORY	STATUS
WEEK OF OCT 12			
Mon, Oct 12	AIDL vs HIDL: Writing a modern HAL in AOSP	HAL	Planned
Thu, Oct 15	Bringing up a new board: a practical BSP checklist	BSP	Planned
WEEK OF OCT 19			
Mon, Oct 19	What changes for vendors in Android 16 GKI	Kernel	Planned

STATUS	MEANING
Planned	In the plan, not written yet.
Written	A blog post has been written from it, not posted yet.
Posted	Its blog is published.
Missed	The date has passed and it isn't posted.
Cancelled	Taken off the schedule. Uploading it again brings it back.

This week counts Monday to Sunday. Click a row to see the full brief (keywords, tone, length, target versions, reference links). Admins can **Cancel** a blog that isn't posted yet.

5 How it is stored in Supabase

Each blog in the JSON becomes **one row** in the Supabase table `public.blog_plan`.



The browser never writes to Supabase directly; only the backend does, through its database connection. Row level security is on with no policies, so the website's public Supabase key can't read or change the table.

What one row looks like

The `BLG-001` entry from the sample file is saved as:

COLUMN	VALUE
<code>id</code>	<code>3f9c...</code> generated by the database; used in the dashboard's links
<code>external_id</code>	<code>BLG-001</code> , the JSON id (unique)
<code>topic</code>	AIDL vs HIDL: Writing a modern HAL in AOSP
<code>category</code>	HAL
<code>keywords</code>	<code>{AIDL,HIDL,Treble}</code> (a list)
<code>publish_date</code>	2026-10-12
<code>tone</code> / <code>length</code>	technical, beginner-friendly · 1000-1500 words
<code>target_versions</code>	<code>{Android 15}</code>
<code>reference_urls</code>	<code>{https://source.android.com/docs/core/architecture/aidl}</code>
<code>status</code>	<code>planned</code> (or <code>written</code> , <code>posted</code> , <code>cancelled</code>). Missed is worked out from the date, never stored.
<code>post_id</code>	Empty until a blog post is written from it (<code>content_posts.id</code>)
<code>uploaded_by</code>	Email of the admin who last uploaded it
<code>created_at</code> / <code>updated_at</code>	When it was first added and last changed

What changes a row

- **Same id uploaded again:** the row is updated, never duplicated.
- **Posted blog:** never changed by an upload.
- **Cancel:** sets `status` to `cancelled`. Rows are never deleted.

Created automatically

The backend runs `create table if not exists` the first time anyone opens the Blogs page or uploads. Running `002_blog_plan.sql` yourself is optional; doing both is harmless.

Looking at it in Supabase

Table Editor → `blog_plan`, or in the SQL Editor:

```
select external_id, publish_date, topic, category, status
from blog_plan
order by publish_date;
```

6 Database reference

“This repo” is `~/Marketing_Agent/dashboardbackend`; “agent repo” is `~/Marketing-AI-Agent`.

Migrations

FILE	CREATES	NEEDED FOR	RUN BY HAND?
<code>002_blog_plan.sql</code> this repo · migrations/	<code>blog_plan</code> , its index, row level security	The blog plan	Optional: the backend runs the same SQL on first use
<code>001_dashboard_presence.sql</code> this repo · migrations/	<code>dashboard_presence</code>	One sign-in per account, checked on every request	Optional: the backend runs it on first use
<code>0002_core.sql</code> and later agent repo · supabase/migrations/	The agent's tables, including <code>content_posts</code>	Blog posts the plan links to	Already applied; not changed by the blog plan
Supabase Auth built in	<code>auth.users</code> , <code>auth.sessions</code>	Sign-in and roles	Nothing to run

`002_blog_plan.sql` needs `content_posts` to exist first, because `blog_plan.post_id` points to it. It already exists in this project (the agent created it).

Tables the blog plan uses

TABLE	OWNED BY	ACCESS	WHY
<code>public.blog_plan</code>	This repo	<code>Read + write</code>	The plan itself, one row per blog

TABLE	OWNED BY	ACCESS	WHY
<code>public.content_posts</code>	Agent repo	Read	Shows Written / Posted and links to the posted blog (through <code>post_id</code>)
<code>auth.users</code>	Supabase Auth	Read	Your role (only admins upload or cancel) and whether your account was removed
<code>auth.sessions</code>	Supabase Auth	Read	Whether your sign-in session still exists
<code>public.dashboard_presence</code>	This repo	Read + write	One sign-in per account (every request)

What each action does

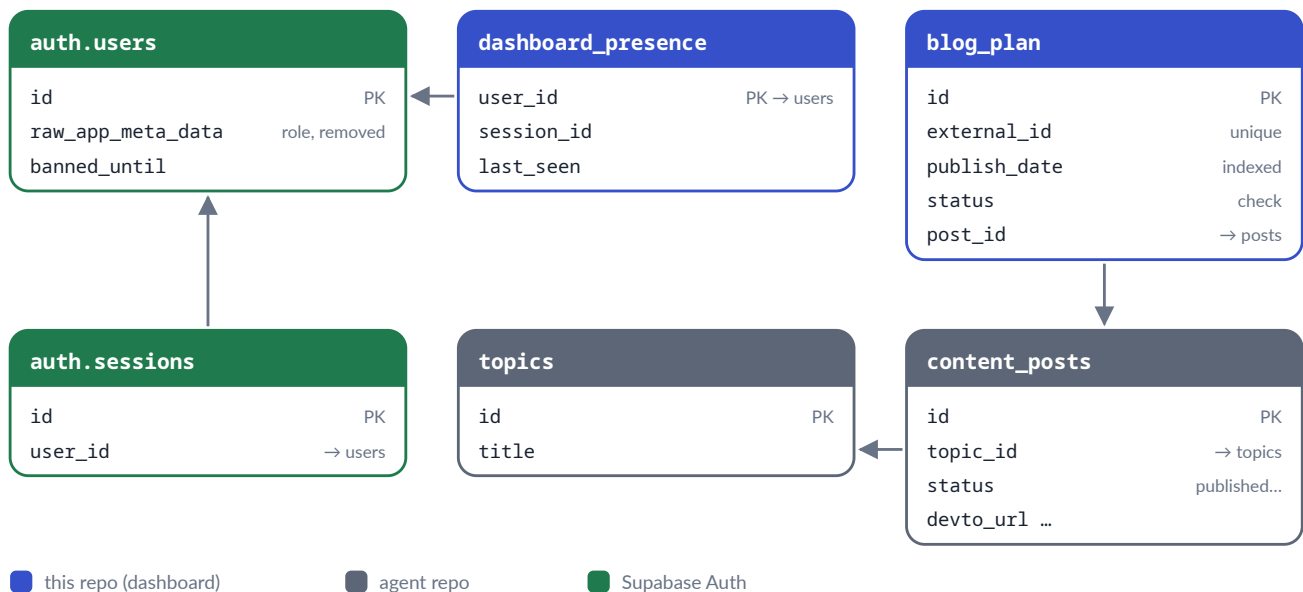
ON THE BLOGS PAGE	BACKEND ROUTE	DATABASE
Open the page	GET <code>/blogs/plan</code>	Reads <code>blog_plan</code> joined with <code>content_posts</code>
Choose a file (the check)	POST <code>/blogs/plan/import</code> <code>?dry_run=true</code>	Reads which ids already exist; writes nothing
Import	POST <code>/blogs/plan/import</code>	One transaction: locks the existing rows, then insert or update each blog by <code>external_id</code> (posted ones skipped)
Click a row	GET <code>/blogs/plan/{id}</code>	Reads one row
Cancel	POST <code>/blogs/plan/{id}/cancel</code>	Sets <code>status = 'cancelled'</code>

Every route also checks your sign-in first (`auth.users` , `auth.sessions` , `dashboard_presence`). Values from the file are always sent as query parameters, never pasted into the SQL.

Rules the database enforces

- `external_id` is **unique**: the same JSON id can only exist once.
- `status` must be `planned` , `written` , `posted` or `cancelled` .
- `post_id` must point to a real `content_posts` row; if that post is deleted, `post_id` becomes empty.
- `publish_date` is **indexed**, so sorting the schedule stays fast.
- **Row level security** is on with no policies: only the backend's database connection can use the table.

7 Table schemas



How the tables connect. Arrows point from the column that refers to another table.

public.blog_plan this repo · migrations/002_blog_plan.sql (the whole file)

```
-- Blog plan (app/blog_plan.py, BLOG_PLAN.md): one row per blog brief uploaded as JSON on the Blogs
page.
-- The backend also creates this on first use; running it here by hand is optional. Run once in the
Supabase SQL editor.
create table if not exists public.blog_plan (
  id                uuid primary key default gen_random_uuid(),
  external_id       text not null unique,           -- the JSON "id", e.g. BLG-001
  topic             text not null,
  category          text not null,
  keywords          text[] not null default '{}',
  publish_date      date not null,
  tone              text,
  length            text,
  target_versions   text[] not null default '{}',
  reference_urls    text[] not null default '{}',
  status            text not null default 'planned'
                  check (status in ('planned', 'written', 'posted', 'cancelled')),
  post_id           uuid references public.content_posts (id) on delete set null, -- the blog written
  from it
  uploaded_by       text,                          -- email of the admin who uploaded it
  created_at        timestampz not null default now(),
  updated_at        timestampz not null default now()
);
create index if not exists blog_plan_publish_date_idx on public.blog_plan (publish_date);

-- Only the dashboard backend (database connection) uses this table. RLS with no policies keeps it out
of reach of
-- the browser's anon / authenticated keys.
alter table public.blog_plan enable row level security;
```

public.dashboard_presence this repo · migrations/001_dashboard_presence.sql (the whole file)

```
-- One sign-in per account (app/auth.py): which session is using each account, and when it was last
seen.
-- Run once in the Supabase SQL editor.
create table if not exists public.dashboard_presence (
  user_id      uuid primary key references auth.users (id) on delete cascade,
  session_id   uuid not null,
  last_seen    timestampz not null default now()
);

-- Only the dashboard backend (database connection) uses this table. RLS with no policies keeps it out
of reach of
-- the browser's anon / authenticated keys.
alter table public.dashboard_presence enable row level security;
```

public.content_posts agent repo · 0002_core.sql + 0003_daily_agent.sql, as it is now

The blog plan only reads **id**, **status** and the three URL columns. A post's **published** status shows as **Posted** ; any other status of a linked post shows as **Written** .

```
create table content_posts (
  id                uuid primary key default gen_random_uuid(),
  topic_id          uuid references topics (id),
  title             text,
  body_md           text,
  status            text not null default 'idea'
                    check (status in ('idea', 'drafted', 'in_review', 'approved',
                                      'published', 'rejected', 'publish_failed')),
  reviewer_note     text,
  cms_url           text,
  devto_url         text,
  linkedin_url      text,
  idempotency_key   text unique,          -- 'blog-YYYY-MM-DD' for the agent's daily blog
  metrics           jsonb,
  embedding          vector(1024),
  embedding_model    text,
  metadata          jsonb,               -- added in 0003_daily_agent.sql
  tags              text[],              -- added in 0003_daily_agent.sql
  created_at        timestampz not null default now(),
  updated_at        timestampz not null default now(), -- set by trigger touch_updated_at
  published_at      timestampz
);
create index content_posts_status_idx on content_posts (status);
-- Row level security on; signed-in team members may read (policy content_posts_team_read).
```

auth.users and **auth.sessions** Supabase Auth · managed by Supabase

No migration of ours changes these. The columns the backend reads:

TABLE	COLUMN	USED FOR
auth.users	id	The signed-in person (the token's sub)
auth.users	raw_app_meta_data	role = admin allows upload and cancel; removed blocks a removed account
auth.users	banned_until	A banned account is refused
auth.sessions	id, user_id	Whether the token's sign-in session still exists

8 Uploading again

- A blog with the same `id` is **updated**: topic, date, keywords and the rest are replaced.
- A blog whose post is already **published** is never changed and shows as Skipped.
- A **cancelled** blog uploaded again goes back to Planned.
- Blogs in the database but not in the new file are left as they are. To remove one, cancel it.

9 What is not automatic yet

Nothing writes a blog from a brief or publishes it on its date yet. `post_id` and `status` are there so that can be added. The options:

OPTION 1

The agent writes it

The agent's daily blog (`agent/blog.py`) picks its own topic today. It could take the next brief due within a few days, write the post into `content_posts`, and set `post_id` and `status = 'written'`.

OPTION 2

A job on Render

A scheduled job publishes posts whose brief's date has come, then sets `status = 'posted'`.

OPTION 3

By hand

Write and post the blog yourself, then set `post_id` so the dashboard shows it as posted.

With option 1, decide whether the agent should stop picking its own topics, or both will produce blogs.

10 Testing it

FILE (IN DASHBOARDBACKEND/SAMPLES/)	WHAT IT DOES
<code>blog_plan.sample.json</code>	6 valid blogs, Mondays and Thursdays from Oct 12 to Oct 29, 2026. Upload it: the preview shows 6 New , and after Import the card shows 6 planned blogs. Upload it again: 6 Update .
<code>blog_plan.invalid.json</code>	One mistake per blog: missing topic, impossible date, past date, <code>ftp://</code> link, duplicate id, empty keywords, unusual category. Every problem is listed and Import stays disabled.

The sample dates are fixed. After Oct 29, 2026 they are in the past and the valid file will be refused; change the dates to test again.

Automated tests: `pytest tests/test_blog_plan.py` covers validation with both sample files, import, dry run, admin-only access, cancel, the statuses and the summary counts.